

Research Challenge: Entity Search

Camila Nicola
Departamento de Ciência da
Computação, UFMG
Belo Horizonte, Minas Gerais, Brasil

Samira Malaquias
Departamento de Ciência da
Computação, UFMG
Belo Horizonte, Minas Gerais, Brasil

Victoria Estanislau
Departamento de Ciência da
Computação, UFMG
Belo Horizonte, Minas Gerais, Brasil

1 Introduction

Entity retrieval is the task of identifying the most relevant entities for a given query from a large collection of structured or semi-structured data. In this challenge, each query must be answered by ranking entities from a large corpus, where each entity is represented by fields such as title, keywords, and descriptive text. This setting requires balancing exact lexical matching with broader semantic relevance, since some queries directly mention entity names or categories, while others describe entities indirectly or use vocabulary that differs from the corpus.

Our approach follows an incremental experimental strategy. We start with a purely lexical BM25 baseline, using field weighting, rank fusion, and handcrafted reranking heuristics to establish a strong and interpretable sparse retrieval system. We then extend this baseline with a supervised Learning-to-Rank model, testing whether LambdaMART can learn better combinations of the same lexical and entity-specific signals than manually defined weights.

After evaluating the limits of lexical retrieval, we move toward hybrid ranking. The next submission adds dense semantic features to address vocabulary mismatch, while still using BM25 as the scalable first-stage candidate generator. We then add a cross-encoder reranking signal to model query-entity interactions more directly for the strongest candidates. Finally, we replace the smaller cross-encoder with a stronger instruction-tuned Qwen4B reranker, aiming to improve relevance estimation for ambiguous and descriptive queries.

Overall, the report is organized as a progression from simple and interpretable sparse retrieval to increasingly expressive hybrid and neural reranking pipelines: lexical BM25 baseline, lexical LTR, dense-enhanced LTR, cross-encoder-enhanced LTR, and Qwen4B-enhanced LTR. This structure allows us to analyze not only which submission achieved the best score, but also how each added component changed the effectiveness and limitations of the previous system.

2 Methodology

2.1 Lexical BM25 Baseline

Submission: `submission.csv` **Kaggle score:** 0.43145

2.1.1 Hypothesis. The hypothesis behind this submission was that a simple lexical retrieval pipeline could provide a strong initial baseline for the entity search task. Since many entity queries contain terms that appear directly in the entity title, keywords, or description, a carefully tuned BM25-based system should be able to retrieve useful rankings even without supervised learning.

This run tested how far a lightweight sparse retrieval strategy could go using only lexical matching, field weighting, rank fusion,

and heuristic reranking. The goal was to establish a robust baseline that could later be used to evaluate whether more complex strategies, such as Learning-to-Rank, larger candidate pools, query expansion, dense features, or neural rerankers, actually improved effectiveness.

2.1.2 Indexing Strategy. This submission used a SQLite FTS5 index built over the structured entity corpus. Each entity was indexed using its main textual fields: title, keywords, and descriptive text. The retrieval strategy relied on multiple field-weighted BM25 variants, assigning different levels of importance to each field.

Some variants emphasized the title, based on the assumption that title matches are often strong indicators of entity relevance. Others gave more weight to keywords, which are useful when the query describes a topic or category rather than naming the entity directly. The descriptive text was also indexed to improve coverage, although with lower importance because longer fields can introduce noisy matches.

The index also included Porter-stemmed variants to reduce mismatches caused by morphological variation. One of these variants also removed simple English stopwords from the query, reducing the influence of very common terms.

2.1.3 Query Processing and Ranking Strategy. Query processing was entirely lexical. Queries were lowercased and tokenized with a simple word-based tokenizer. Most retrieval variants used the OR operator, prioritizing recall by allowing entities to match even when they contained only part of the query. In one Porter-stemmed variant, common English stopwords were removed before retrieval.

Each query was executed against multiple BM25 configurations, including a balanced field-weighted variant, a title-heavy variant, a title-plus-keywords-heavy variant, a keywords-heavy variant, a Porter-stemmed balanced variant, and a Porter-stemmed title-plus-keywords variant with stopword removal.

The rankings produced by these variants were combined using Reciprocal Rank Fusion (RRF). This rewarded entities that appeared in multiple rankings, while still preserving strong evidence from individual BM25 configurations. After collecting candidates, the pipeline applied a final heuristic reranking function.

The final score combined the RRF score, best BM25 score, number of variants retrieving the entity, best rank signal, and a handcrafted entity-search bonus. This bonus rewarded query coverage in the title, keywords, and text; bigram matches; phrase occurrence; cases where all query terms appeared in the title or in title+keywords; and short titles with high query coverage. It also penalized long titles with weak query coverage, since these may match the query only incidentally.

2.1.4 Discussion. The main strength of this submission is its simplicity and interpretability. The ranking behavior is easy to understand:

entities are promoted when they match the query in important fields, appear in multiple BM25 variants, and show strong title or keyword coverage. This makes the method useful as a diagnostic baseline for later experiments.

The public score of 0.43145 shows that a purely lexical approach already captures a meaningful amount of relevance. This is expected in entity search, where many queries contain names, categories, or descriptive terms that occur directly in entity titles and keywords. The use of multiple field-weighted BM25 variants improved robustness compared with a single BM25 configuration, since different queries benefit from different fields.

However, this method has clear limitations. Because it does not use relevance judgments, it cannot learn optimal feature weights from training data. The final reranking function is manually designed, so it may overvalue some signals and undervalue others. For example, title coverage is often important, but its exact importance relative to keyword or text coverage should ideally be learned from labeled data.

Another limitation is vocabulary mismatch. The system depends on lexical overlap between the query and the entity fields. If a relevant entity is described using synonyms, paraphrases, or related concepts that do not share surface terms with the query, this approach may fail to retrieve or rank it highly. Porter stemming helps with morphological variation, but it does not address semantic mismatch.

Overall, this submission established a strong first baseline: a sparse BM25 retrieval system with field weighting, RRF, and entity-specific heuristics. Its main advantage was robustness with low complexity, while its weaknesses motivated later submissions using supervised LambdaMART reranking, larger candidate pools, query expansion, dense semantic features, and neural reranking.

2.2 LTR v2 k=1000

Submission: submission_ltr_v2_k1000.csv **Kaggle score:** 0.44017

2.2.1 Hypothesis. After establishing a strong lexical baseline with BM25 variants, RRF, and handcrafted reranking heuristics, the next question was whether these signals could be combined more effectively by a supervised Learning-to-Rank model. The hypothesis behind this submission was that a trained reranker could learn better weights for lexical and entity-specific matching features than the manually designed scoring function used in the previous submission.

This run also increased the first-stage candidate pool to candidate_k=1000. In a two-stage retrieval pipeline, the reranker can only reorder entities retrieved during candidate generation. Therefore, increasing the candidate pool was expected to improve recall and give the supervised model more opportunities to promote relevant entities into the final top-100 ranking. The expected trade-off was that a larger pool would introduce more noisy candidates, but that LambdaMART would be able to learn how to demote them.

2.2.2 Indexing Strategy. The indexing strategy remained fully lexical and based on SQLite FTS5, making this submission a direct extension of the previous sparse retrieval baseline. The corpus

was indexed through multiple field-based views, including title, keywords, text, title+keywords, and all.

As in the baseline, this organization allowed the system to exploit the different roles of each field. The title field provides a precise signal for direct entity matches, keywords offer a compact topical representation, and text improves coverage at the cost of additional noise. Porter-stemmed variants, such as docs_title_keywords_porter and docs_all_porter, were also included to reduce mismatches caused by morphological variation. No dense embeddings or neural encoders were used at this stage, so any improvement over the previous submission would come from supervised reranking and a larger candidate set rather than from semantic representation learning.

2.2.3 Query Processing and Ranking Strategy. Queries were normalized by lowercasing, removing accents, and applying a simple tokenizer. Each query was executed against several BM25 variants, differing in indexed field, boolean operator, and contribution to the lexical ensemble.

Compared with the previous baseline, this submission used the same general principle of combining complementary lexical views, but with a larger candidate pool and a supervised second stage. More restrictive variants used the AND operator over fields such as title, title+keywords, keywords, and all, favoring high-precision matches. Less restrictive variants used OR, improving recall when only part of the query matched the entity.

The rankings produced by these BM25 variants were combined using Reciprocal Rank Fusion. After fusion, the system retained up to 1000 candidates per query. These candidates were then reranked using a LightGBM LambdaMART model trained on the available relevance judgments.

The reranker used both retrieval-based and entity-specific features. Retrieval features included RRF score, best rank, mean rank, best BM25 score, number of variants retrieving the entity, and individual rank and BM25 values for each BM25 variant. Entity-specific matching features included query coverage in title, keywords, and text; Jaccard overlap; bigram coverage; phrase occurrence; exact title match; ordered title matching; field lengths; and penalties for long titles with low query coverage.

2.2.4 Discussion. This submission improved the public score from 0.43145 to 0.44017, suggesting that supervised reranking was beneficial over the manually designed lexical baseline. The gain is consistent with the hypothesis that LambdaMART can learn a better combination of relevance signals than fixed heuristic weights. For instance, the model can learn when title coverage should dominate, when keyword overlap is more informative, and when matches in long text fields should be treated as weaker evidence.

The larger candidate pool also made the pipeline less dependent on the top results of any single BM25 configuration. This is important because entity search is highly sensitive to first-stage recall: if a relevant entity is not retrieved initially, the reranker cannot recover it. By keeping up to 1000 candidates, the system increased the chance that relevant entities were available to the second stage.

However, the improvement over the baseline was moderate, which reveals two important limitations. First, the approach was still entirely lexical, so it remained vulnerable to vocabulary mismatch, synonyms, and paraphrases. Second, increasing

candidate_k also introduced many weak candidates, making the reranking task harder. The supervised model helped filter this noise, but it could not compensate for relevant entities that were still missed by lexical retrieval.

Overall, this submission represents the first supervised extension of the lexical baseline. It kept the same sparse retrieval foundation, but replaced the final handcrafted ranking logic with LambdaMART and expanded the candidate pool. Its improved score motivated later submissions that further extended the same pipeline with query expansion, model ensembles, dense semantic features, and neural reranking.

2.3 LTR v3 Dense

Submission: submission_ltr_v3_dense.csv **Kaggle score:** 0.50457

2.3.1 Hypothesis. After the previous submission showed that supervised reranking improves over a handcrafted lexical baseline, the next limitation to address was vocabulary mismatch. The LTR v2 pipeline was still mostly dependent on exact or stemmed term overlap between the query and the entity fields. Therefore, it could fail when a relevant entity was semantically related to the query but described with different words.

The hypothesis behind this submission was that adding dense semantic features to the V3 Learning-to-Rank pipeline would improve ranking quality over a purely lexical LTR system. Dense embeddings were expected to provide semantic evidence that BM25 and lexical matching features could not capture. However, the dense model was not used to replace lexical retrieval. Instead, BM25 remained responsible for efficient candidate generation, while dense similarities were added as features for the supervised reranker.

2.3.2 Indexing Strategy. The first-stage index remained lexical and was implemented with SQLite FTS5. The corpus was indexed through multiple field-based views, including title, keywords, title+keywords, text, and all, as well as Porter-stemmed variants. The system also used legacy field-weighted BM25 indexes, where title, keywords, and text received different weights.

No full dense index was built over the complete corpus. This design choice kept retrieval computationally feasible, since the corpus contains millions of entities. Instead, BM25 retrieved a candidate pool of up to 1000 entities per query, and dense similarities were computed only for those candidates.

The dense encoder used in this submission was sentence-transformers/all-MiniLM-L6-v2. For each candidate entity, the system encoded four textual representations: title, keywords, title+keywords, and title+keywords+short text. This allowed the reranker to compare the query with both compact and richer descriptions of each entity.

2.3.3 Query Processing and Ranking Strategy. Queries were processed using the V3 lexical pipeline. Each query was executed against several BM25 variants with different fields, operators, stemming choices, and field weights. Compared with the previous LTR v2 run, this version also incorporated global query expansion learned from the training relevance judgments. Expanded terms were added to selected retrieval variants to improve recall when the original query terms were insufficient.

The candidate lists produced by the BM25 variants were combined using Reciprocal Rank Fusion, and the top 1000 candidates were passed to the reranking stage. Each candidate was then represented by both lexical and dense features.

The lexical features included RRF score, best rank, mean rank, best BM25 score, number of variants retrieving the entity, field-specific query coverage, Jaccard overlap, phrase matches, bigram matches, exact title match, title and keyword length, heuristic bonus, expansion features, and per-variant BM25 and rank values.

The dense features measured cosine similarity between the query embedding and each candidate representation. These included similarity to title, keywords, title+keywords, and title+keywords+short text, as well as aggregate signals such as maximum dense similarity, mean dense similarity, dense rank, and inverse dense rank.

The final ranking was produced by an ensemble of three LightGBM LambdaMART models with different hyperparameters and random seeds. The ensemble averaged the model predictions to reduce variance and make the final ranking more robust. This submission used dense features and ensemble mode, but did not use a cross-encoder.

2.3.4 Discussion. This submission improved the public score from 0.44017 to 0.50457, representing a substantial gain over the previous lexical LTR configuration. This result supports the hypothesis that dense semantic features provide useful additional evidence for entity ranking. While BM25 captures exact lexical correspondence, the dense encoder helps identify candidates that are semantically related to the query even when surface overlap is limited.

An important strength of this approach is that it preserves the efficiency and precision of lexical retrieval. BM25 remains responsible for searching the full corpus and generating candidates, while the dense model is applied only to a much smaller candidate set. This makes the method more practical than full dense retrieval over millions of entities, while still allowing the reranker to use semantic similarity signals.

The LambdaMART ensemble also helps control the risk of noisy dense matches. A candidate may be semantically close to the query but not actually relevant to the intended entity search task. By combining dense similarities with lexical, structural, and field-specific features, the model can learn when semantic evidence should be trusted and when lexical evidence should dominate.

The main limitation is that dense features are computed only for entities already retrieved by the lexical candidate generator. Therefore, if BM25 fails to retrieve a relevant entity in the top 1000, the dense model cannot recover it. In addition, all-MiniLM-L6-v2 is a compact general-purpose encoder, so it may not capture fine-grained entity relevance distinctions as well as a larger or task-specific neural model.

Overall, this submission represents a clear transition from purely lexical LTR to a hybrid lexical-semantic ranking pipeline. It keeps BM25 and RRF as a strong candidate generation backbone, but enriches the reranker with dense semantic evidence and model ensembling. The large improvement in public score motivated the next step: using a cross-encoder to directly estimate query-entity relevance for the strongest candidates.

2.4 LTR v3 Cross-Encoder

Submission: `submission_ltr_v3_cross.csv` **Kaggle score:** 0.50746

2.4.1 Hypothesis. After the previous submission showed that dense semantic features improved the lexical LTR pipeline, the next step was to test whether a stronger neural matching signal could further improve precision. The dense V3 submission used bi-encoder similarities, where the query and entity representations are encoded independently. Although this is efficient, it may miss fine-grained interactions between the query terms and the entity fields.

The hypothesis behind this submission was that adding a cross-encoder signal on top of the dense V3 pipeline would improve final ranking quality. Unlike bi-encoders, a cross-encoder jointly processes the query and the entity text, allowing the model to estimate their relevance more directly. This was expected to be especially useful for distinguishing candidates that are lexically or semantically similar but differ in actual relevance.

2.4.2 Indexing Strategy. The indexing strategy was the same as in the V3 dense submission. Candidate generation remained fully lexical and used SQLite FTS5 indexes over multiple field-based views, including title, keywords, title+keywords, text, and all, as well as Porter-stemmed variants. The system also used legacy field-weighted BM25 variants, assigning different weights to title, keywords, and text.

No dense or cross-encoder index was built over the complete corpus. Neural models were applied only after sparse retrieval, which kept the system computationally feasible for a corpus with millions of entities. Thus, BM25 and RRF still provided the candidate generation backbone, while dense and cross-encoder scores were used as reranking features.

2.4.3 Query Processing and Ranking Strategy. Query processing followed the V3 pipeline. Queries were normalized and submitted to multiple BM25 variants with different fields, operators, stemming configurations, and field weights. The system also used global query expansion learned from the training relevance judgments, with expanded terms added to selected retrieval variants to improve lexical recall.

The first retrieval stage kept `candidate_k=1000`. The candidate lists produced by the BM25 variants were combined using Reciprocal Rank Fusion. For each candidate, the system computed the same V3 lexical features used in the dense submission, including RRF score, best rank, mean rank, best BM25 score, number of variants retrieving the entity, field-specific term coverage, Jaccard overlap, phrase and bigram matches, exact title match, title length, heuristic bonus, expansion features, and per-variant BM25 and rank values.

The pipeline also kept the dense features from sentence-transformers/all-MiniLM-L6-v2. These features included cosine similarities between the query and several candidate representations, such as title, keywords, title+keywords, and title+keywords+short text.

The main addition in this submission was a cross-encoder feature block. The model used was cross-encoder/ms-marco-MiniLM-L-6-v2. For each query, the cross-encoder was applied

to the top candidates selected according to dense similarity, with `cross_encoder_top_k=200`. Each input pair combined the query with a textual representation of the entity, using the title, keywords, and the beginning of the descriptive text.

The resulting cross-encoder score, cross-encoder rank, inverse rank, score difference relative to dense similarity, and an indicator of whether the candidate was scored by the cross-encoder were added as features. The final ranking was produced by an ensemble of three LightGBM LambdaMART models, which combined lexical, dense, and cross-encoder features and averaged predictions across different hyperparameters and random seeds.

2.4.4 Discussion. This submission improved the public score from 0.50457 to 0.50746. Although the gain over the dense-only configuration was modest, it indicates that the cross-encoder added useful relevance information beyond independent embedding similarity. This result is consistent with the hypothesis that jointly encoding the query and entity text can capture more precise matching patterns.

The main advantage of the cross-encoder is that it evaluates query-entity compatibility more directly. Dense cosine similarity can identify candidates that are broadly related to the query, but it may not determine whether the candidate satisfies the exact information need. The cross-encoder can model interactions between query terms and entity fields, helping distinguish candidates that appear similarly relevant according to lexical and dense evidence.

However, the improvement was limited for several reasons. First, the previous dense V3 system was already strong, combining lexical features, query expansion, dense similarities, and model ensembling. Second, the cross-encoder was applied only to the top 200 candidates selected after earlier stages. If a relevant entity was not included in this subset, the cross-encoder could not improve its ranking. Third, cross-encoder/ms-marco-MiniLM-L-6-v2 is a compact model trained for passage ranking, not specifically for this entity search task, so its relevance estimates may not always align with the competition labels.

The main drawback of this submission was computational cost. Cross-encoder inference is much more expensive than BM25 or bi-encoder similarity because each query-entity pair must be processed jointly. This cost was controlled by applying the model only to the top 200 candidates and using its output as features rather than as a full-corpus ranking method.

Overall, this submission extended the hybrid lexical-semantic pipeline with a more precise neural interaction signal. The positive but small improvement suggests that cross-encoders are useful mainly for difficult cases where candidates are already close according to lexical and dense evidence. This result motivated the final direction of exploring stronger neural relevance estimation with larger language models.

2.5 LTR v3 Qwen4B Reranker

Submission: `submission_ltr_v3_qwen4b.csv` **Kaggle score:** 0.53561

2.5.1 Hypothesis. The previous submission showed that adding a cross-encoder signal improved the dense V3 pipeline, although the gain was modest. One possible explanation was that the model used,

cross-encoder/ms-marco-MiniLM-L-6-v2, was relatively small and trained mainly for passage ranking rather than for structured entity search. Therefore, the next step was to test whether a stronger instruction-tuned reranker could provide a more accurate query-entity relevance signal.

The hypothesis behind this submission was that Qwen/Qwen3-Reranker-4B would improve final ranking quality by better modeling the relation between short natural-language queries and structured entity descriptions. This was expected to be especially useful for ambiguous or descriptive queries, where lexical overlap, dense cosine similarity, and a compact cross-encoder may not be sufficient to identify the most relevant entity.

2.5.2 Indexing Strategy. The indexing strategy remained the same as in the V3 pipeline. Candidate generation used SQLite FTS5 lexical indexes over multiple field-based views, including title, keywords, title+keywords, text, and all, as well as Porter-stemmed variants. The system also used legacy field-weighted BM25 variants with different weights for title, keywords, and text.

No full neural index was built over the corpus. BM25 and RRF remained responsible for retrieving a candidate pool of up to 1000 entities per query. The Qwen reranker was applied only after sparse candidate generation and dense feature computation. This preserved the scalability of lexical retrieval over millions of entities while allowing a stronger neural model to influence the final ranking.

2.5.3 Query Processing and Ranking Strategy. This submission reused the V3 pipeline with lexical features, query expansion, dense features, and a LambdaMART ensemble. Queries were normalized and searched using multiple BM25 variants with different fields, operators, stemming choices, and field weights. Global query expansion, learned from the training relevance judgments, added related terms to selected retrieval variants to improve lexical recall.

The candidate lists were combined using Reciprocal Rank Fusion. For each candidate, the pipeline computed the same lexical features used in previous V3 submissions, including RRF score, best rank, mean rank, best BM25 score, number of variants retrieving the entity, field-specific term coverage, Jaccard overlap, phrase and bigram matches, exact title match, title length, heuristic bonus, expansion features, and per-variant rank and BM25 values.

Dense features were also computed with sentence-transformers/all-MiniLM-L6-v2. These features measured cosine similarity between the query and multiple candidate representations: title, keywords, title+keywords, and title+keywords+short text.

The main change was implemented in `ltr_pipeline_v3_qwen4b.py`, which wraps the V3 pipeline and replaces the default cross-encoder with Qwen/Qwen3-Reranker-4B. Each candidate was formatted as a structured entity document containing its title, keywords, and a truncated description. The reranker received the query and the formatted entity text, together with an instruction prompt asking it to judge whether the entity was relevant to the entity-search query.

The Qwen reranker was applied to the top 200 candidates per query, selected from the dense ranking. Its output was converted into cross-encoder-style features: reranker score, reranker rank, inverse rank, score difference relative to dense similarity, and an indicator of whether the candidate received a neural score. The final ranking was produced by an ensemble of three LightGBM

LambdaMART models using lexical, dense, and Qwen reranker features.

2.5.4 Discussion. This was the strongest submission, reaching a public score of 0.53561. The improvement over the previous cross-encoder run, from 0.50746 to 0.53561, suggests that the quality of the neural reranking signal had a substantial impact on final effectiveness. While the MiniLM cross-encoder added useful information, the larger Qwen reranker produced a more informative relevance signal for this task.

The main advantage of this approach was stronger query-entity matching. Unlike bi-encoder dense features, the Qwen reranker evaluates the query and the candidate entity together. Compared with the smaller cross-encoder, it likely provides better language understanding and captures descriptive queries, entity attributes, and indirect relevance. The structured prompt also helped adapt the model by framing each input as an entity relevance judgment.

This was particularly useful when lexical and dense evidence were ambiguous. Two candidates may share query terms or have similar embeddings, but only one may satisfy the actual information need. By considering the title, keywords, and description together, the Qwen reranker can provide a more precise relevance score, which the LambdaMART ensemble then combines with lexical and dense signals.

The main limitation was computational cost. A 4B reranker is much more expensive than BM25, dense cosine similarity, or the smaller MiniLM cross-encoder. For this reason, the model was applied only to the top 200 candidates and its scores were cached. As a consequence, it could not help entities outside this reranking subset, and the whole pipeline still depended on the lexical candidate generator to retrieve relevant entities early.

Another limitation is that the Qwen reranker was used as a feature generator rather than being fine-tuned on the competition relevance judgments. Its scores may not perfectly match the dataset labels, but they still provided a strong enough signal to improve the final LambdaMART ranking.

Overall, this submission completed the progression from sparse lexical retrieval to a hybrid lexical-semantic-neural ranking pipeline. The best performance came from combining scalable BM25 candidate generation, RRF, query expansion, dense semantic features, and a high-quality instruction-tuned reranking signal inside an ensemble Learning-to-Rank model.